

Práctica 3: ECUACIONES NO LINEALES

Objetivos

1. Trabajar computacionalmente las nociones vistas en Enseñanzas Básicas correspondientes a resolución de ecuaciones no lineales, utilizando MATHEMATICA.
2. Conocer los comandos MATHEMATICA que permitan realizar el tratamiento computacional anteriormente indicado.

Contenidos

1. Comando NSolve

El comando `NSolve[ecuación, incógnita]` permite obtener aproximaciones numéricas de las soluciones de cualquier ecuación polinómica.

```
In[1] := NSolve[x3-2x2+5x-4==0, x]
Out[1] = {{x -> 0.5+1.93649I}, {x -> 0.5-1.93649I}, {x -> 1.}}
```

Este comando también permite resolver sistemas de ecuaciones polinómicas, teniendo en cuenta que tanto las ecuaciones como las incógnitas van entre llaves y separadas por comas:

```
In[2] := NSolve[{x2+y2+3x*y==0, 3x2-5x*y+4y==0}, {x, y}]
Out[2] = {{x -> 0.650841, y -> -1.70392}, {x -> 0.311185, y -> -0.118862},
          {x -> 0., y -> 0.}, {x -> 0., y -> 0.}}
```

Las soluciones devueltas por el comando tienen 6 cifras (significativas) por defecto. Para aumentar el número de cifras devueltas en las soluciones, utilizaremos la opción `WorkingPrecision`, que determina la precisión de trabajo para el comando:

```
In[3] := NSolve[x3-2x2+5x-4==0, x, WorkingPrecision -> 12]
Out[3] = {{x -> 0.500000000000+1.93649167310I},
          {x -> 0.500000000000-1.93649167310I}, {x -> 1.000000000000}}
```

Aunque el comando `NSolve` está pensado para resolver ecuaciones polinómicas, también da aproximaciones para las soluciones de otras muchas ecuaciones no polinómicas, aunque en este caso no nos asegura que las que devuelve sean todas las que existen y eso es lo que significa el mensaje de advertencia que devuelve al ser ejecutado el comando.

```

In[4]:= NSolve[E^x==x,x]
InverseFunction::ifun : Inverse functions are being used. Values may
      be lost for multivalued inverses.
Solve::ifun :
      Inverse functions are being used by Solve, so some solutions may
      not be found; use Reduce for complete solution information
Out[4]= {{x→0.318132 - 1.33724 I}}

```

Pero incluso puede dar un error de tal modo que no sea capaz de dar ninguna solución de la ecuación, independientemente de que dicha ecuación tenga solución o no. Por ejemplo, la ecuación $\cos(x) = x$ tiene una solución en el intervalo $(0, 1)$, pero la orden `NSolve` devuelve:

```

In[5]:= NSolve[Cos[x]==x,x]
Solve::tdep :
      The equations appear to involve the variables to be solved for
      in an essentially non-algebraic way
Out[5]= NSolve[Cos[x]==x,x]

```

2. Comando FindRoot

Para resolver numéricamente ecuaciones de cualquier tipo (polinómicas o no), podemos hacer uso del comando `FindRoot[ecuación, {x, x0}}`. Con este comando se obtiene la solución numérica de la ecuación tomando como iterado inicial a $x=x_0$, mediante un método basado en el de Newton. Veámoslo para la ecuación que nos daba problemas con el comando `NSolve`:

```

In[6]:= FindRoot[Cos[x]==x, {x, 1}]
Out[6]= {x→0.739085}

```

En caso de introducir dos estimaciones iniciales, el comando `FindRoot` utiliza una variante del método de la secante, no requiriendo de derivadas para obtener nuevos iterados:

```

In[7]:= FindRoot[Cos[x]==x, {x, 1, 2}]
Out[7]= {x→0.739085}

```

Para modificar la precisión de respuesta en el comando `FindRoot`, solo es necesaria la opción `WorkingPrecision`. De este modo, para disponer de aproximaciones de 20 cifras significativas, se escribirá la opción `WorkingPrecision→20`.

```

In[8]:= FindRoot[Cos[x]==x, {x, 1}, WorkingPrecision→20]
Out[8]= {x→0.73908513321516062512}

```

El comando `FindRoot` también se utiliza para resolver sistemas de ecuaciones (polinómicas o no):

```

In[9]:=FindRoot[{x^2+y^2+3x*y==0,3x^2-5x*y+4y==0},{x,-1},{y,1}]
Out[9]= {{x→0.311185,y→-0.118862}}
In[10]:=FindRoot[{Exp[x-2]==y,y^2==x},{x,1},{y,1}]
Out[10]= {x→0.019026,y→0.137935}

```

Al igual que cuando estábamos trabajando con una única ecuación, el comando `FindRoot` aplicado a sistemas admite que en vez de dar solo un iterado inicial para cada incógnita se den dos. Eso sí, todas las incógnitas deben venir dadas por un dato inicial o todas por dos. Si en unas incógnitas solo se da un dato y en otras se dan dos, el comando no es capaz de realizar los cálculos y falla.

```

In[11]:=FindRoot[{Exp[x-2]==y,y^2==x},{x,2,3},{y,1,2}]
Out[11]= {{x→2.44754,y→1.56446}}
In[12]:=FindRoot[{Exp[x-2]==y,y^2==x},{x,2,3},{y,1}]
FindRoot::fdin:
  Form of start specification {y,1} in FindRoot[{Exp[x-2]==y,y^2==x},
    {{x,2,3},{y,1}}] is inconsistent with other start specifications
Out[12]= FindRoot[{Exp[x-2]==y,y^2==x},{x,2,3},{y,1}]

```

Para finalizar, indicar que el comando `FindRoot` admite tanto ecuaciones como funciones en su primer argumento. Es decir, podríamos aplicarle el comando a la ecuación $\cos(x) = x$ o a la función $f(x) = \cos(x) - x$.

```

In[13]:=FindRoot[Cos[x]-x,{x,1},WorkingPrecision→20]
Out[13]={x→0.73908513321516062512}

```

Problemas a realizar

Ejercicio 1. ¿Cuántas soluciones tiene la ecuación $x^3 - x^2 - 4x + 3 = 0$? Halla intervalos de longitud 0.1 que contengan a cada una de ellas y calcula la mayor de estas soluciones con un error absoluto inferior a $\frac{1}{2} \cdot 10^{-3}$ aplicando el método de bisección.

Ejercicio 2. Elige un intervalo en el que podamos asegurar la existencia de una única solución positiva de la ecuación $2x = \tan(x)$. ¿Cuántas iteraciones serían necesarias por el método de bisección para hallar una aproximación de dicha solución con una exactitud de 5 cifras decimales exactas? Utilizando la misma tolerancia, obtén una aproximación de dicha solución mediante el método de la secante. Determina, si es posible, la velocidad de convergencia del método de la secante en el intervalo considerado

Ejercicio 3. Considera la ecuación $x^4 + 3x^2 - 2 = 0$ con $x \in [-4, 4]$.

1. Localiza y separa las soluciones de la ecuación anterior.
2. Determina una aproximación de cada solución con una precisión de 5 cifras significativas usando el método de Regula-Falsi.
3. ¿Existen soluciones de la ecuación en los intervalos $(-\infty, -4)$ y $(4, +\infty)$?

Ejercicio 4. Dada la ecuación $x = e^{-x}$, responde a los siguientes apartados:

1. Localiza y separa las soluciones de la ecuación. Indica cuántas hay en total. Demuestra que existe una única solución en el intervalo $[0, 1]$.
2. Calcula la solución existente en el intervalo $[0, 1]$ utilizando los métodos de bisección, de la secante y de regula falsi. En todos los casos, la aproximación obtenida debe tener un error absoluto menor que 10^{-6} .
3. Determina, si es posible, la velocidad de convergencia del método de la secante en el intervalo considerado.

Problemas de ampliación

Ejercicio 5. Resuelve la ecuación $e^x - \log(x+2) - 2 = 0$ por medio de los métodos de bisección y de la secante, sabiendo que tiene dos soluciones en el intervalo $(-2, +\infty)$ y que solo una de ellas es positiva. La solución debe tener 5 cifras decimales exactas.

Ejercicio 6. Calcula una aproximación con tres cifras decimales exactas para la única solución de la ecuación $x^5 + 2x^3 - 1 = 0$ en el intervalo $[0, 1]$. Utiliza para ello el método de regula falsi y, al menos, precisión 5.

Ejercicio 7. La ecuación $e^{-x} + 4x^3 - 5 = 0$ posee una única solución en el intervalo $[1, 2]$. Justifica que puede aplicarse el método de bisección y realiza las primeras ocho iteraciones del mismo para obtener una aproximación de la solución. Estima una cota del error absoluto para la aproximación obtenida en cada iteración. Indica cuál debería ser el número de iteraciones mínimo para asegurar que el número de cifras decimales exactas es 10.

Ejercicio 8. Localiza y separa las soluciones de la ecuación $x^3 - 3x^2 - 4x + 6 = 0$. Los intervalos usados deben ser de amplitud inferior a 0.5. Aplica el método de la secante para obtener una aproximación de la solución más grande con error relativo menor que $3 \cdot 10^{-6}$. Determina la velocidad de convergencia del método en los intervalos considerados.